

Homework - 8

① Initialization

$$W = 13$$

Priority queue - Pq

$$\text{maxprofit} = 0$$

$$P_i = \{20, 30, 35, 12, 3\}$$

$$W_i = \{2, 5, 7, 3, 1\}$$

→ Step 0 : Root node

$$\text{weight } w = 0, \text{ profit } P = 0$$

$$\text{bound} = 20 + 30 + \frac{6}{7} \times 35 = 80$$

$$\text{insert root into Pq} = [\text{Node}(\text{root}, \text{bound} = 80)]$$

→ Step 1 : process root node

• remove root from Pq.

$$\text{bound} = 80 > \text{maxprofit} = 0$$

• include item 1 : level = 1, $w = 0 + 2 = 2$, $P = 0 + 20 = 20$

$$w = 2 \leq W = 13 \rightarrow \text{we update maxprofit} = 20.$$

$$\text{bound is still same } 20 + 30 + 30 = 80$$

$$80 > 20, \text{ insert into Pq.}$$

• exclude 1 : level = 1, $w = 0$, $P = 0$

$$\text{bound} = 30 + 35 + \frac{1}{3} \times 12 = 69$$

$$\text{bound (69)} > \text{max profit } 20 \rightarrow \text{we insert into Pq}$$

$$\text{Pq} = [\text{Node}(\text{in 1}, \text{bound} = 80), \text{Node}(\text{ex 1}, \text{bound} = 69)]$$

→ Step-2 : process Node(in1, bound=80)

- Remove from P2, $80 > 20 \rightarrow$ branch on item 2.

- include item 2 : level = 2, $w = 2+5=7$, $p = 50$

$w = 7 \leq W = 13 \rightarrow$ update maxprofit = 50

bound = $50 + \frac{6}{7} \times 35 = 80 > 50 \rightarrow$ insert into P2.

- exclude 2 : level = 2, $w = 2$, $p = 20$

bound = $20 + 35 + 12 + 3 = 70$

bound 70 > max profit 50 $\rightarrow$ we insert into P2

- $P2 = \left[\text{Node}(\text{in1}, \text{in2}, \text{bound} = 80), \text{Node}(\text{in1}, \text{ex2}, \text{bound} = 70), \right.$
 $\left. \text{Node}(\text{ex1}, \text{bound} = 69) \right]$

→ Step-3 : process Node(in1, in2, bound=80)

- include item 3 : level = 3, $w = 7+7=14$, $p = 85$

$w = 14 > W = 13 \rightarrow$ don't update maxprofit & prune it.

- exclude 3 : level = 3, $w = 7$, $p = 50$

bound = $50 + 12 + 3 = 65 > 50 \rightarrow$ insert into P2

- $P2 = \left[\text{Node}(\text{in1}, \text{ex2}, \text{bound} = 70), \text{Node}(\text{ex1}, \text{bound} = 69), \right.$
 $\left. \text{Node}(\text{in1}, \text{in2}, \text{ex3}, \text{bound} = 65) \right]$

→ Step-4 : process Node(in1, ex2, bound=70)

- remove it from P2 since bound=70 > maxprofit(50) $\rightarrow$ branch on item3

- include item 3 : level = 3, $w = 2+7=9$, $p = 20+35=55$

$w = 9 \leq W = 13 \rightarrow$ update maxprofit = 55

bound = $55 + 12 + 3 = 70$

70 > max profit (55) $\rightarrow$ insert into P2

- exclude 3: level=3, $w = 2$, $p = 20$

bound = $20 + 12 + 3 = 35 < \text{max profit } (55) \rightarrow \text{prune it}$

- $PQ = [\text{Node}(\text{in1}, \text{ex2}, \text{in3}, \text{bound} = 70), \text{Node}(\text{ex1}, \text{bound} = 69), \text{Node}(\text{in1}, \text{in2}, \text{ex3}, \text{bound} = 65)]$

→ Step-5: process Node(in1, ex2, in3, bound = 70)

- remove from PQ since bound = 70 > max profit (55) → branch on item 4

- include 4: level=4, $w = 9 + 3 = 12$, $p = 55 + 12 = 67$.

$w = 12 \leq W = 13 \rightarrow \text{update max profit} = 67$

bound = $67 + 3 = 70 > \text{max profit } (67) \rightarrow \text{insert into PQ}$

- exclude 4: level=4, $w = 9$, $p = 55$

bound = $55 + 3 = 58 < \text{max profit } (67) \rightarrow \text{prune it}$

- $PQ = [\text{Node}(\text{in1}, \text{ex2}, \text{in3}, \text{in4}, \text{bound} = 70), \text{Node}(\text{ex1}, \text{bound} = 69), \text{Node}(\text{in1}, \text{in2}, \text{ex3}, \text{bound} = 65)]$

→ Step-6: process Node(in1, ex2, in3, in4, bound = 70)

- remove from PQ since bound = 70 > max profit (67) → branch on item 5

- include 5: level=5, $w = 12 + 1 = 13$, $p = 67 + 3 = 70$

$w = 13 \leq W = 13 \rightarrow \text{update max profit} = 70$

bound = 70 → no items to add.

$70 > \text{max profit } (70) \rightarrow \text{don't insert into PQ}$

- exclude 5: level=5, $w = 12$, $p = 67$

bound = $67 < \text{max profit } (70) \rightarrow \text{don't insert into PQ}$

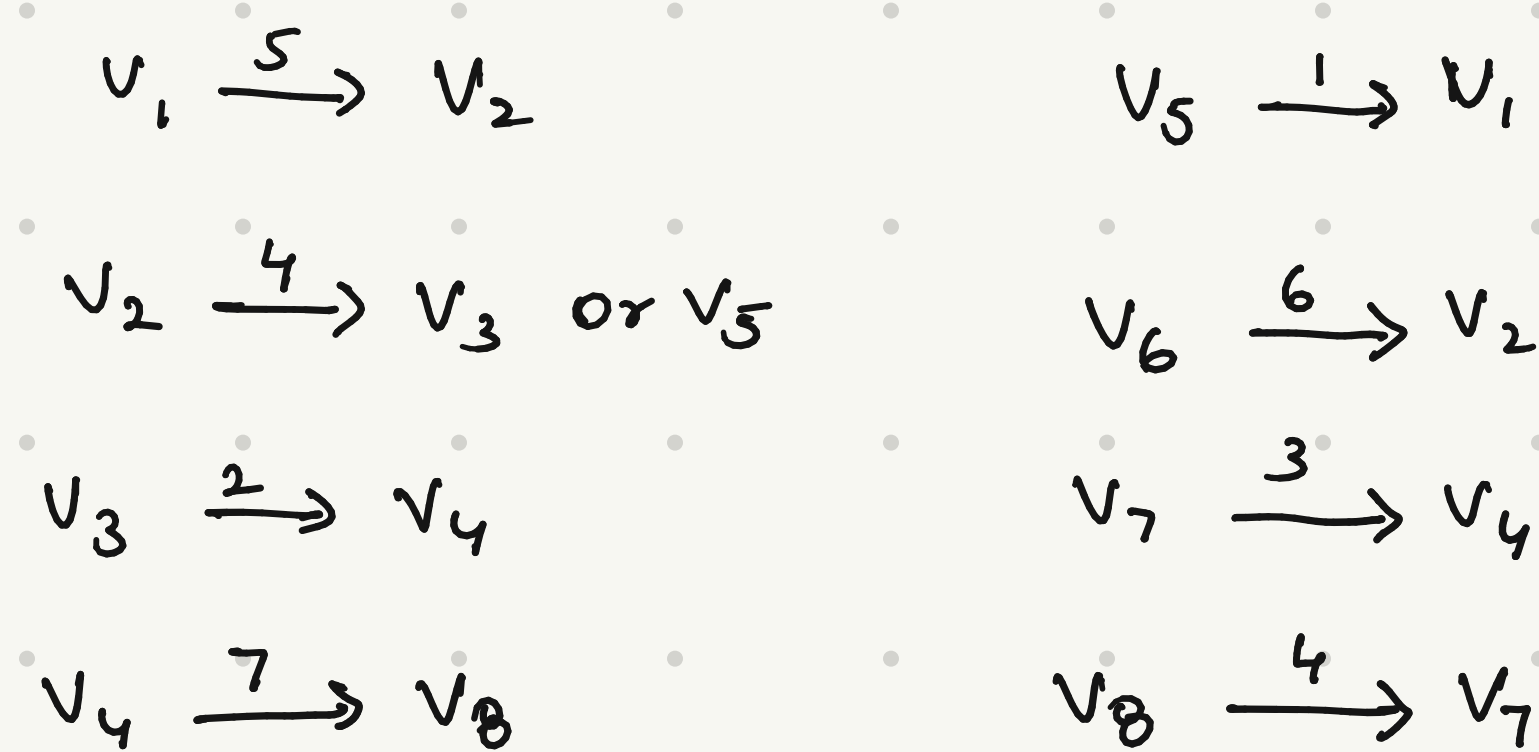
- $PQ = [\text{Node}(\text{ex1}, \text{bound} = 69), \text{Node}(\text{in1}, \text{in2}, \text{ex3}, \text{bound} = 65)]$

→ Step 7 : process remaining nodes

- Node(ex1, bound = 69) . bound $69 < 70 \rightarrow$ we prune it
- Node(in1, in2, ex3, bound = 65) .
bound = $65 < \text{max profit}(70) \rightarrow$ we prune it.
- P2 is empty and algorithm completes.

∴ maxprofit = 70 with item $(P_i, W_i) = \{(20, 2), (35, 7), (12, 3), (3, 1)\}$.

② Minimum outgoing edges from each vertex



initial bound = $5 + 4 + 2 + 7 + 1 + 6 + 3 + 4 = 28$

minlength = ∞

• at each new branch

bound = parent bound - min-out(current node) + actual edge cost

• we always expand the node with the minimum bound in the priority queue (P2).

→ At root node [1] bound = 28

• child [1, 2] : bound = $28 - 5 + 5 = 28$

• child [1, 3] : bound = $28 - 5 + 8 = 31$

$P2 = \{(1, 2, \text{bound} = 28), (1, 3, \text{bound} = 31)\}$

→ Remove $(1, 2, \text{bound} = 28)$

• child $[1, 2, 3] : \text{bound} = 28 - 4 + 4 = 28$

• child $[1, 2, 5] : \text{bound} = 28 - 4 + 4 = 28$

$p_2 = \{ (1, 2, 3, \text{bound} = 28), (1, 2, 5, \text{bound} = 28), (1, 3, \text{bound} = 31) \}$

→ Remove $(1, 2, 3, \text{bound} = 28)$

• child $[1, 2, 3, 4] : \text{bound} = 28 - 2 + 2 = 28$

• child $[1, 2, 3, 7] : \text{bound} = 28 - 2 + 5 = 31$

$p_2 = \{ (1, 2, 3, 4, \text{bound} = 28), (1, 2, 5, \text{bound} = 28), (1, 3, \text{bound} = 31), (1, 2, 3, 7, \text{bound} = 31) \}$

→ Remove $(1, 2, 3, 4, \text{bound} = 28)$

• child $[1, 2, 3, 4, 8] : \text{bound} = 28 - 7 + 7 = 28$

$p_2 = \{ (1, 2, 3, 4, 8, \text{bound} = 28), (1, 2, 5, \text{bound} = 28), (1, 3, \text{bound} = 31), (1, 2, 3, 7, \text{bound} = 31) \}$

→ Remove $(1, 2, 3, 4, 8, \text{bound} = 28)$

• child $[1, 2, 3, 4, 8, 6] : \text{bound} = 28 - 4 + 5 = 29$

• child $[1, 2, 3, 4, 8, 7] : \text{bound} = 28 - 4 + 4 = 28$

$p_2 = \{ (1, 2, 3, 4, 8, 7, b = 28), (1, 2, 5, b = 28), (1, 2, 3, 4, 8, 6, b = 29), (1, 3, b = 31), (1, 2, 3, 7, b = 31) \} \quad (b = \text{bound})$

→ Remove $(1, 2, 3, 4, 8, 7, b = 28)$

• child $[1, 2, 3, 4, 8, 7, 6] : \text{bound} = 28 - 3 + 8 = 33$

$p_2 = \{ (1, 2, 5, b = 28), (1, 2, 3, 4, 8, 6, b = 29), (1, 3, b = 31), (1, 2, 3, 7, b = 31), (1, 2, 3, 4, 8, 7, 6, b = 33) \}$

→ Remove $(1, 2, 5, b = 28)$

- no other nodes to go from v_5 . This is a dead end.

Hence, this path can be pruned.

$$P_2 = \{ (1, 2, 3, 4, 8, 6, b = 29), (1, 3, b = 31), (1, 2, 3, 7, b = 31), \\ (1, 2, 3, 4, 8, 7, 6, b = 33) \}$$

→ Remove $(1, 2, 3, 4, 8, 6, b = 29)$

- child $[1, 2, 3, 4, 8, 6, 5]$: bound = $29 - 2 + 2 = 29$

$$P_2 = \{ (1, 2, 3, 4, 8, 6, 5, b = 29), (1, 3, b = 31), (1, 2, 3, 7, b = 31), \\ (1, 2, 3, 4, 8, 7, 6, b = 33) \}$$

→ Remove $(1, 2, 3, 4, 8, 6, 5, b = 29)$

- cannot reach v_7 from v_5 . This is a dead end.

Hence, this path is pruned.

$$P_2 = \{ (1, 2, 3, 7, b = 31), (1, 3, b = 31), (1, 2, 3, 4, 8, 7, 6, b = 33) \}$$

→ Remove $(1, 2, 3, 7, b = 31)$

- child $[1, 2, 3, 7, 4]$: bound = $31 - 3 + 3 = 31$

- child $[1, 2, 3, 7, 6]$: bound = $31 - 3 + 8 = 36$

$$P_2 = \{ (1, 2, 3, 7, 4, b = 31), (1, 3, b = 31), (1, 2, 3, 4, 8, 7, 6, b = 33), \\ (1, 2, 3, 7, 6, b = 36) \}$$

→ Remove $(1, 2, 3, 7, 4, b = 31)$

- child $[1, 2, 3, 7, 4, 8]$: bound = $31 - 7 + 7 = 31$

$$P_2 = \{ (1, 2, 3, 7, 4, 8, b = 31), (1, 3, b = 31), (1, 2, 3, 4, 8, 7, 6, b = 33), \\ (1, 2, 3, 7, 6, b = 36) \}$$

→ Remove $(1, 2, 3, 7, 4, 8, b = 31)$

• Child $[1, 2, 3, 7, 4, 8, 6] : \text{bound} = 31 - 4 + 5 = 32$

$p_2 = \{ (1, 3, b = 31), (1, 2, 3, 7, 4, 8, 6, b = 32), (1, 2, 3, 4, 8, 7, 6, b = 33), (1, 2, 3, 7, 6, b = 36) \}$

→ if we pick $(1, 3)$ its children will generate bounds higher than 31 and leading to bound 36 paths. So, we will pick $(1, 2, 3, 7, 4, 8, 6, b = 32) \rightarrow \text{node}$.

→ Remove $(1, 2, 3, 7, 4, 8, 6, b = 32)$

• Child $[1, 2, 3, 7, 4, 8, 6, 5] : 32 - 2 + 2 = 32$

→ From v_5 we can reach v_1 . Hence the travel is complete.

Path = $[1, 2, 3, 7, 4, 8, 6, 5, 1]$

$32 < \text{minlength}(\infty) \rightarrow \text{update } \text{minlength} = 32$

Since, other paths have bounds > 32 they are pruned and algorithm is complete.

∴ Optimal Path = $1 \rightarrow 2 \rightarrow 3 \rightarrow 7 \rightarrow 4 \rightarrow 8 \rightarrow 6 \rightarrow 5 \rightarrow 1$

Length = 32

⑤ Three more applications of this

a) Finding the most cost-effective way to assign a set of workers to equal number of tasks, where each worker has different cost for each task.

b) We can use this to efficiently process tasks on multiple machines to minimize overall completion time.

c) We can use it for optimization problem in discrete domain. Maximize a linear objective function, subject to constraints and solutions must be whole numbers.